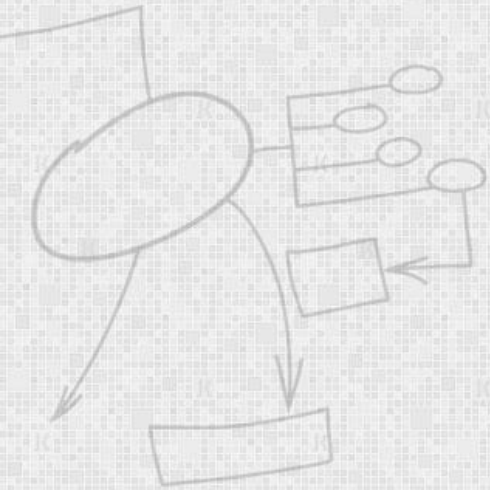
EC-360
SOFTWARE ENGINEERING



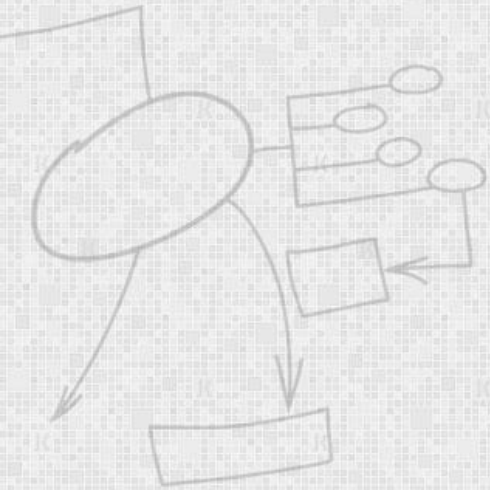
Lecture 10

Coding

By
Dr Wasi Haider Butt

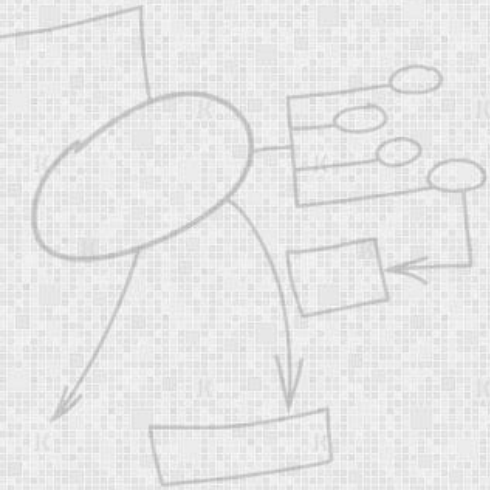
Today's Lecture: Agenda

- Coding Standards
- Benefits
- Example Standards
- Obfuscation



Why STANDARDS

- “Don't patch bad code - rewrite it."
- "Don't stop with your first draft."

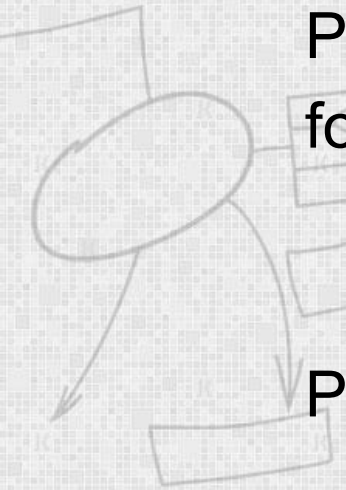


I think there may be a bug in Joe's Code - Please Fix

```
func GreenEggsNHam(Not SamIAm, Green EggsNHam)
    foreach Green TryThem in SamIAm
        do EatThem(TryThem) = false
    NotInACarNotOnABus(EggsNHam)
func NotInACarNotOnABus(Green EggsNHam)
    EatThem(EggsNHam) = true
    NotOnAPlane(EggsNHam)
    foreach NotLikeThem SamIAm of EggsNHam do
        if not EatThem(SamIAm) then
            NotInACarNotOnABus(SamIAm)
    IDoNotLikeThem(EggsNHam)
```

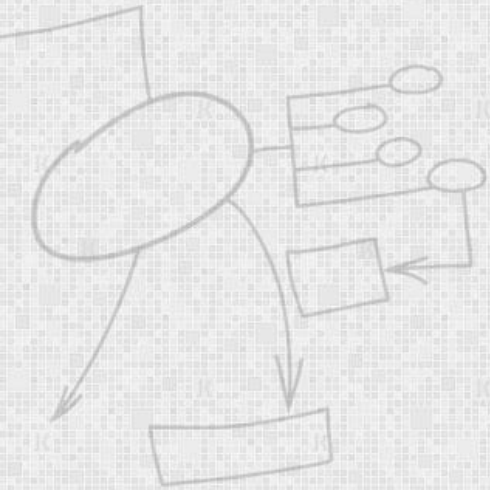
Joe's Code Following a Sane Coding Standard . . .

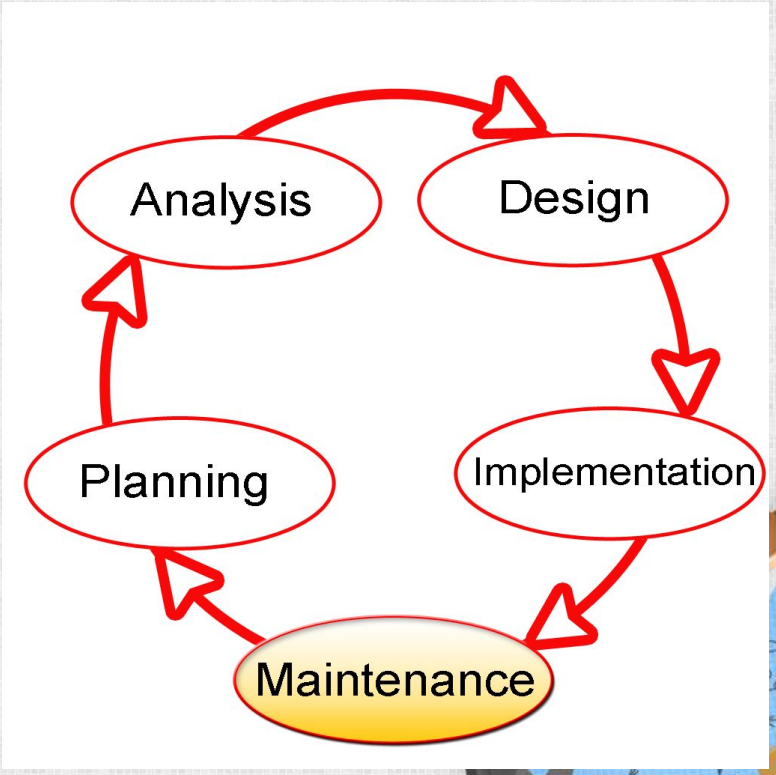
```
func DepthFirstSearch(graph G, vertex v)
    foreach vertex w in G
        do Encountered(w) = false
    RecursiveDFS(v)
func RecursiveDFS(vertex v)
    Encountered(v) = true
    PreVisit(v)
    foreach neighbor w of v do
        if not Encountered(w) then
            RecursiveDFS(w)
    PostVisit(v)
```



Why Have Coding Standards?

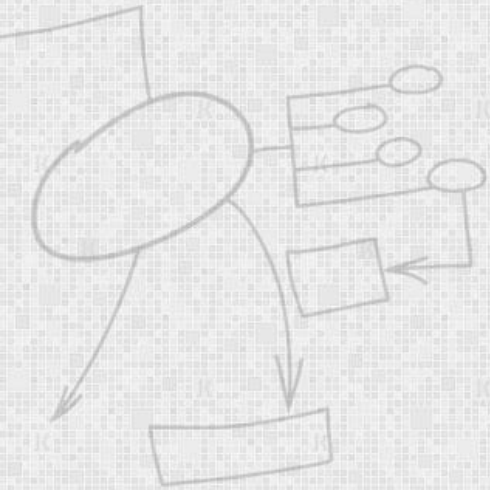
- Software Maintenance
- Less Bugs
- Team switching





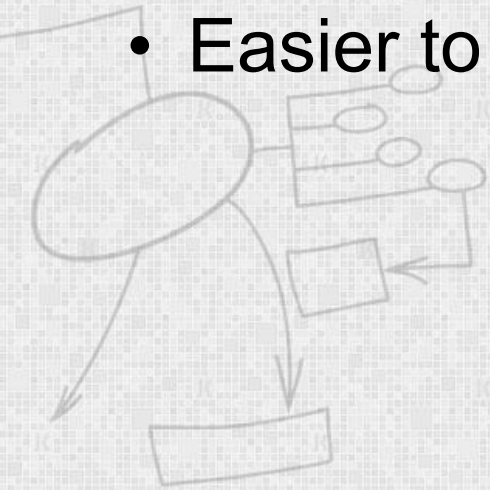
Organizational Req

- Good software development organizations normally require their programmers to adhere to some well-defined coding standards.



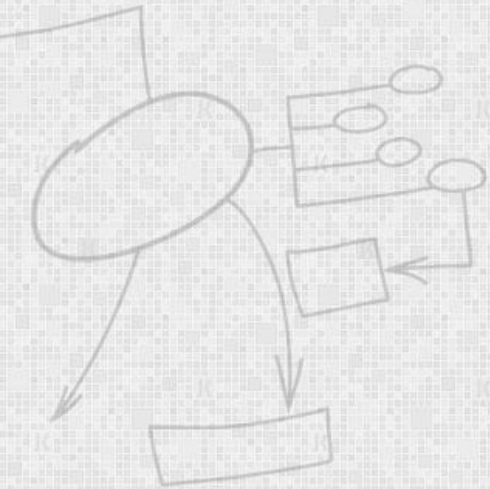
Coding Standards Benefits

- A coding standard gives a uniform appearance to the codes written by different engineers.
- It enhances code understanding.
- Greater consistency between developers
- Easier to develop and maintain



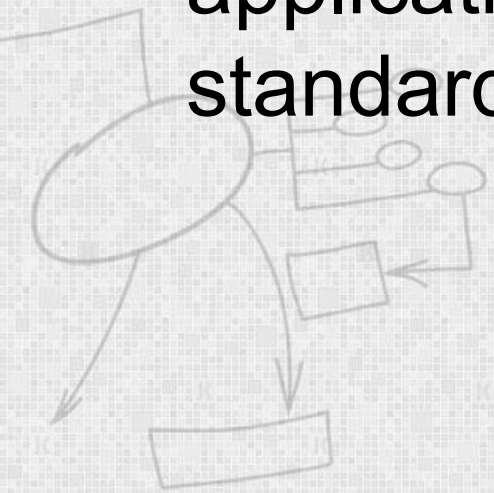
Areas Typically Covered

- Program Design
- Naming Conventions
- Formatting Conventions
- Documentation



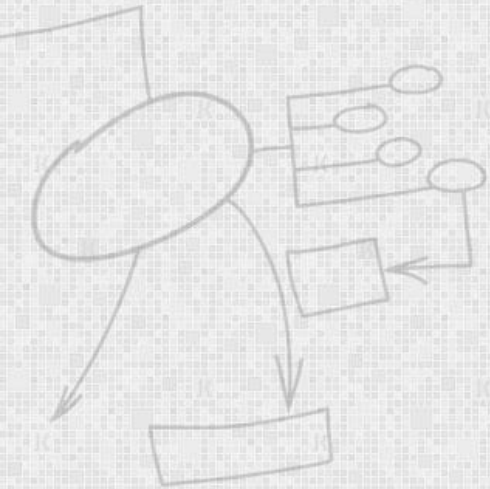
Prime Directive

- **Document every time you violate a standard.**
- No standard is perfect for every application, but failure to comply with your standards requires a comment



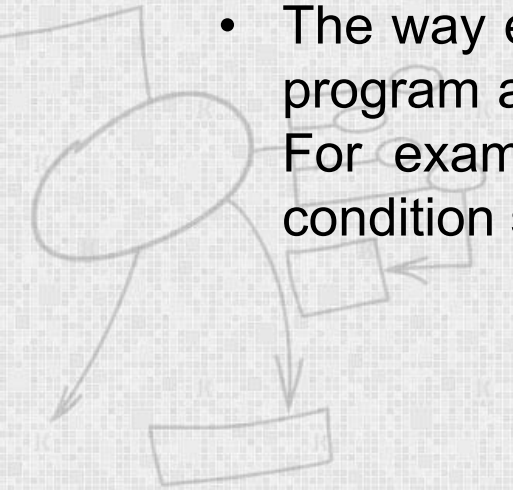
Standards Preference

- **Industry Standards > organizational standards > project standards > personal standards > no standards**



Example Standards

- Naming conventions for global variables, local variables, and constant identifiers:
  - A possible naming convention can be that global variable names always start with a capital letter, local variable names are made of small letters, and constant names are always capital letters.
- Error return conventions and exception handling mechanisms:
  - The way error conditions are reported by different functions in a program are handled should be standard within an organization. For example, different functions while encountering an error condition should either return a 0 or 1 consistently.

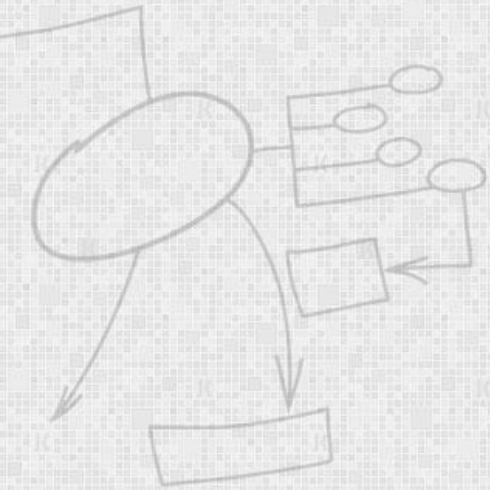


Example Standards

- Do not use an identifier for multiple purposes
  - Programmers often use the same identifier to denote several temporary entities. For example, some programmers use a temporary loop variable for computing and a storing the final result.

Example Standards

- The code should be well-documented:
  - As a rule of thumb, there must be at least one comment line on the average for every three-source line.



Common Practice - Indentation

- Indentation of
  - Functions
  - Objects
  - Etc

- Gives an indication of scope but doesn't affect program

Example

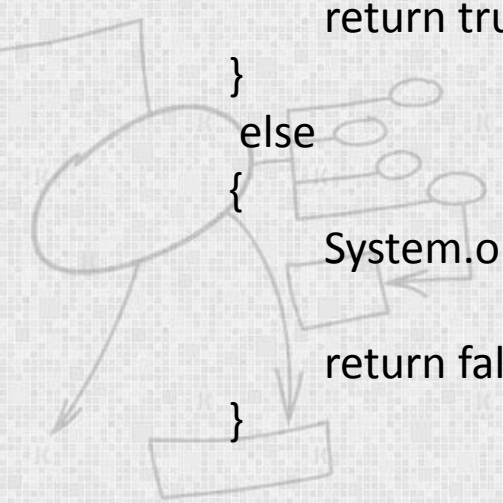
Compare

```
if (g < 17 && h < 22 || i < 60) { return true; } else {System.out.println (“incorrect”) ; return false; }
```

To

```
if (g < 17 && h < 22 ||  i < 60)
{
        return true;
}
 else
{
        System.out.println(“incorrect”);

        return false;
}
```



- Easier to Read

- Easier to Understand

- **Easier to maintain**

Common Practice - Whitespace

- Very important but often overlooked
- Makes the code easier to read
- Especially important with large programs, lack of whitespace can be very confusing

Example

```
for(int i=0;i<40;i++)
        {system.out.println(i);}
```

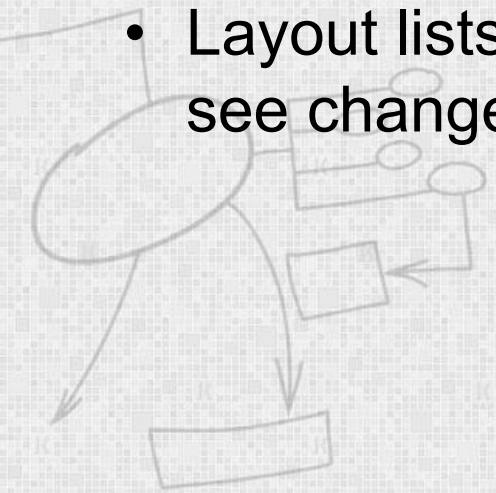
```
for( int i = 0 ; i < 40 ; i++ )
    {
     system.out.println(i);
     }
```

Example recommendations from different coding standards

- Use spaces not TABs.
- Three character indent
- No long lines. Limit the line length to a maximum of 120 characters.
- No trailing whitespace on any line.
- Put brace on a new line.
- Single space around keywords, e.g. if (.
- Single space around binary operators, e.g. 42 + 69
- No space around unary operators, e.g. ++i

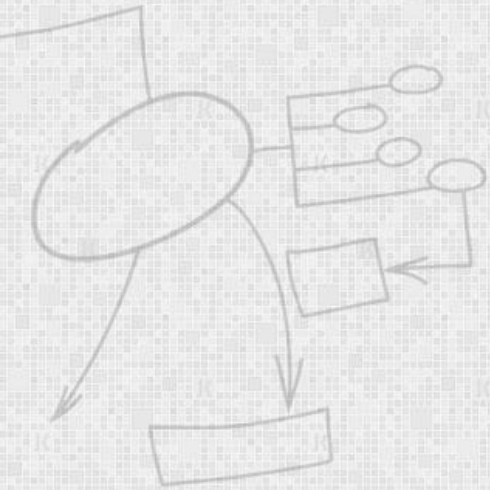
Example recommendations from different coding standards

- No space before parentheses with functions,
  e.g. fred(42, 69)
- Single space after parentheses with functions,
  e.g. fred(42, 69)
- Single space after comma with functions,
  e.g. fred(42, 69)
- Layout lists with one item per line; this makes it easier to see changes in version control.



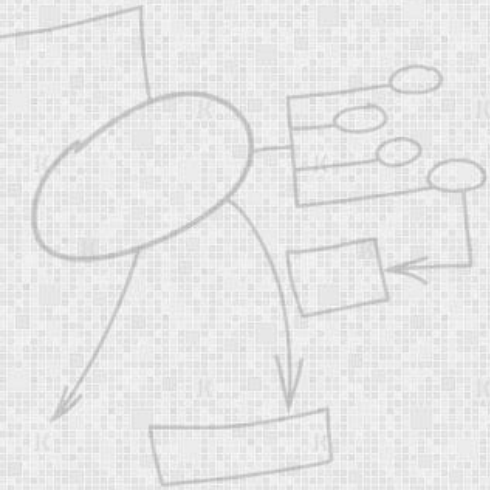
Example quotations from different coding standards

- One declaration per line.
- Avoid big functions and methods. Same for large classes and large files.
- Avoid deep nesting.
- Always use braces with if statements, while loops, etc. This makes changes clearer in version control.



Example

C# Coding Standards and Best Programming Practices



Naming Conventions and Standards

Note :

The terms Pascal Casing and Camel Casing are used in next slides.

Pascal Casing - First character of all words are Upper Case and other characters are lower case.

Example: BackColor

Camel Casing - First character of all words, except the first word are Upper Case and other characters are lower case.

Example: backColor

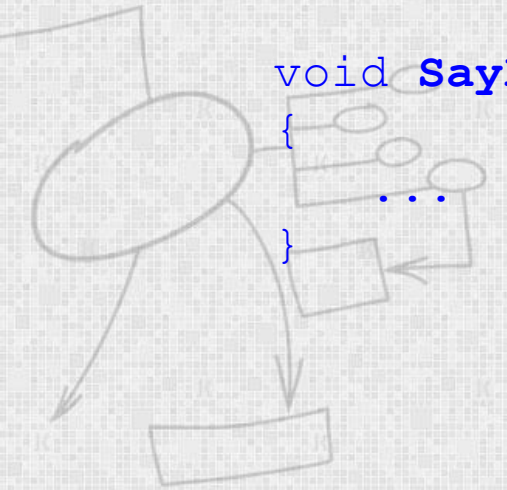
Naming Conventions and Standards

Use Pascal casing for Class names

```
public class HelloWorld
{
    ...
}
```

Use Pascal casing for Method names

```
void SayHello(string name)
{
    ...
}
```

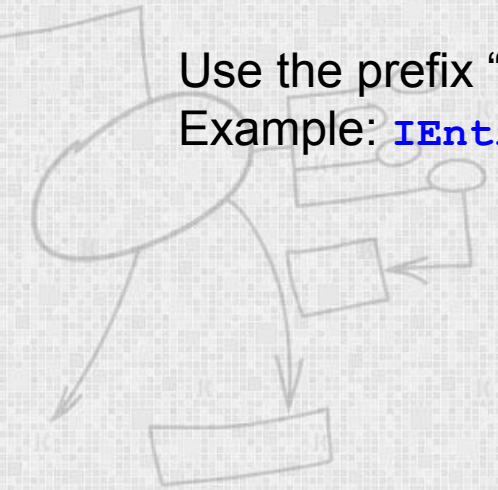


Naming Conventions and Standards

Use Camel casing for variables and method parameters

```
int totalCount = 0;
void SayHello(string name)
{   string fullMessage = "Hello " + name;
    ...
}
```

Use the prefix “I” with Camel Casing for interfaces (Example: `IEntity`)



Naming Conventions and Standards

Use Meaningful, descriptive words to name variables.
Do not use abbreviations.

Good:

```
string address
int salary
```

Not Good:

```
string nam
string addr
int sal
```

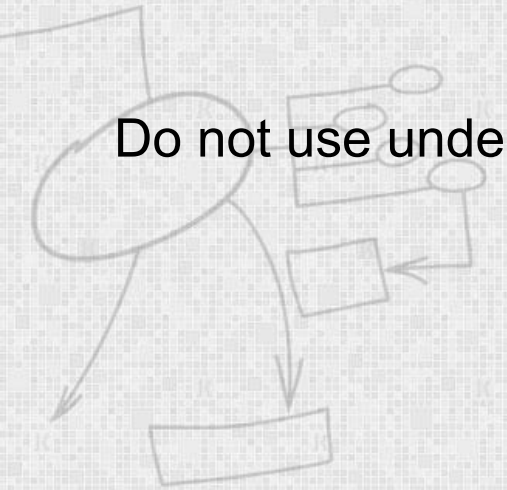
Naming Conventions and Standards

Do not use single character variable names like `i, n, s` etc. Use names like `index, temp`

One exception in this case would be variables used for iterations in loops:

```
for ( int i = 0; i < count; i++ )
{
    ...
}
```

Do not use underscores (_) for local variable names.



Naming Conventions and Standards

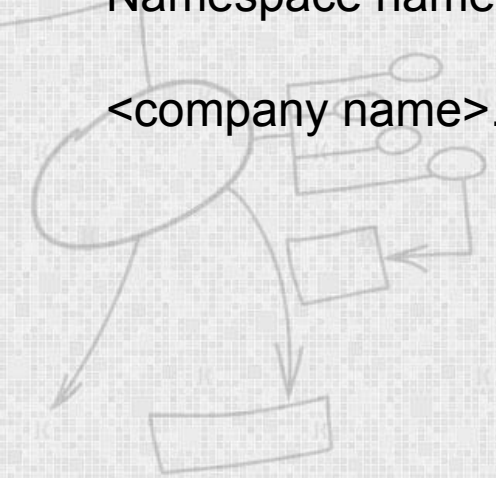
Do not use variable names that resemble keywords.

Prefix `boolean` variables, properties and methods with "`is`" or similar prefixes.

Ex: `private bool _isFinished`

Namespace names should follow the standard pattern

<company name>.<product name>.<top level module>.<bottom level module>



Naming Conventions and Standards

Use appropriate prefix for the UI elements so that you can identify them from the rest of the variables.

There are 2 different approaches recommended here.

a. Use a common prefix (ui_) for all UI elements. This will help you group all of the UI elements together and easy to access all of them from the intellisense.

b. Use appropriate prefix for each of the ui element. A brief list is given below. Since .NET has given several controls, you may have to arrive at a complete list of standard prefixes for each of the controls (including third party controls) you are using.

Naming Conventions and Standards

Control	Prefix
Label	lbl
TextBox	txt
DataGrid	dtg
Button	btn
ImageButton	imb
Hyperlink	hlk
DropDownList	ddl

Naming Conventions and Standards

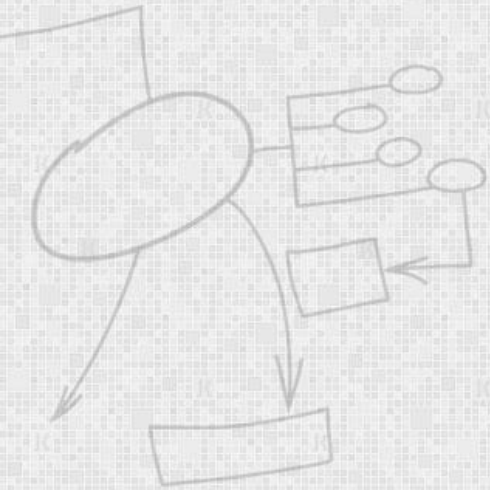
ListBox	lst
DataList	dtl
Repeater	rep
Checkbox	chk
CheckBoxList	cbl
RadioButton	rdo
RadioButtonList	rbl
Image	img
Panel	pnl
PlaceHolder	phd
Table	tbl
Validators	val

Naming Conventions and Standards

File name should match with class name.

For example, for the class HelloWorld, the file name should be helloworld.cs (or, helloworld.vb)

Use Pascal Case for file names.



Indentation and Spacing

- Use `#region` to group related pieces of code together. If you use proper grouping using `#region`, the page should like this when all definitions are collapsed.

```
using System;

namespace EmployeeManagement
{
    public class Employee
    {
        Private Member Variables

        Private Properties

        Private Methods

        Constructors

        Public Properties

        Public Methods
    }
}
```

Good Programming Practice

Avoid writing very long methods. A method should typically have 1~25 lines of code. If a method has more than 25 lines of code, you should consider re factoring into separate methods.

Method name should tell what it does. Do not use mis-leading names. If the method nar is obvious, there is no need of documentation explaining what the method does.

Good:

```
void SavePhoneNumber ( string phoneNumber )
{
    // Save the phone number.
}
```

Not Good:

```
// This method will save the phone number.
void SaveDetails ( string phoneNumber )
{
    // Save the phone number.
}
```

Good Programming Practice

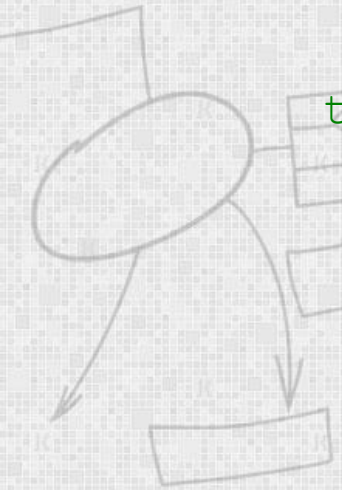
A method should do only 'one job'. Do not combine more than one job in a single method, even if those jobs are very small.

Good:

```
    // Save the address.
    SaveAddress (  address );

    // Send an email to the supervisor to inform
that the address is updated.

    SendEmail ( address, email );
```



Code Obfuscation

Code obfuscation is the act of deliberately obscuring source code, making it very difficult for humans to understand, and making it useless to hackers who may have hidden motives. it may also be used to deter the reverse-engineering of software.